# DESARROLLO WEB CLIENTE - 2.º DAW

## UD 03: Document Object Model

DOM

---

**Material de estudio completo: DOM, manipulación del documento, compatibilidad, React, useRef y enrutamiento**

**Curso 2026-2027**

**Elaborado por: José Fernández González**

**Versión 1.3 · octubre de 2026**

**CC BY-NC-ND 4.0**

**https://github.com/jstechg/material-docente/tree/main/2DAW_Desarrollo_Web_Cliente**

## Índice de estudio

1. DOM, BOM y relación entre window, document y navegador
2. Árbol DOM: nodos, elementos y relaciones
3. Tipos de nodo y propiedades fundamentales
4. Recorrido del árbol: hijos, padres y hermanos
5. Selección de elementos: API clásica y selectores CSS
6. Colecciones DOM: NodeList, HTMLCollection y Array
7. Lectura y modificación de texto y contenido
8. Creación e inserción de nodos
9. Mover, sustituir, eliminar y clonar elementos
10. Atributos HTML, propiedades DOM y data-*
11. CSS desde JavaScript: style, className y classList
12. Carga del documento y ubicación del código JavaScript
13. Eventos del DOM: fundamentos
14. Propagación y delegación de eventos
15. Seguridad al generar contenido dinámico
16. Compatibilidad entre navegadores y desarrollo multicliente
17. Separación de contenido, presentación y comportamiento
18. Patrones útiles
19. React y el DOM: modelo declarativo y reconciliación
20. Listas, datos, map() y key en React
21. useRef y acceso controlado al DOM en React
22. React Router actual: rutas, enlaces y navegación
23. Rutas anidadas, Outlet, parámetros y navegación programática
24. Resumen comparativo para el examen
25. Bibliografía y documentación de consulta

## 1. DOM, BOM y relación entre window, document y navegador

| CONCEPTO | EXPLICACIÓN / QUÉ RECORDAR | EJEMPLO |
|---|---|---|
| **DOM (Document Object Model)** | Representación programable de un documento HTML como un árbol de objetos. JavaScript usa las APIs DOM para consultar y modificar estructura, contenido y estilo. DOM es una API web, no una parte del núcleo de ECMAScript. | `const titulo = document.querySelector("h1");` |
| **BOM (Browser Object Model)** | Nombre informal para APIs del entorno del navegador que rodean al documento: window, location, history, navigator, screen, temporizadores, etc. No existe como una única especificación llamada “BOM”. | `console.log(location.pathname);` |
| **window** | Objeto global de una pestaña/ventana del navegador. Muchas APIs globales son propiedades de window, aunque normalmente se omite el prefijo. | `window.setTimeout(() => console.log("ok"), 500);` |
| **document** | Objeto que representa el documento cargado y da acceso al árbol DOM. Es el punto de entrada habitual para seleccionar y crear elementos. | `document.title = "Panel de aula";` |
| **location** | Describe la URL actual y permite navegar. Sus propiedades ayudan a consultar ruta, parámetros o fragmento. | `console.log(location.href); console.log(location.search);` |
| **history** | Permite avanzar, retroceder o gestionar entradas del historial. Las bibliotecas de routing de SPA se apoyan en la History API. | `history.back();` |
| **navigator** | Aporta información y capacidades del agente de usuario. No debe usarse para “adivinar” el navegador cuando lo que necesitamos es saber si una función está disponible. | `if ("clipboard" in navigator) { /* disponible */ }` |

**ACTUALIZACIÓN** Evita basar la lógica en navigator.appName o navigator.appVersion. La detección de características (feature detection) es más robusta que identificar marca y versión del navegador.

## 2. Árbol DOM: nodos, elementos y relaciones

| CONCEPTO | EXPLICACIÓN / QUÉ RECORDAR | EJEMPLO |
|---|---|---|
| **Árbol lógico** | El documento se organiza jerárquicamente. document contiene el elemento raíz html, que contiene head y body, y éstos contienen otros nodos. | `document.documentElement // <html>` |
| **Nodo (Node)** | Unidad genérica del árbol. Un nodo puede ser un documento, elemento, texto, comentario, etc. “Nodo” es más general que “elemento”. | `document.body instanceof Node // true` |
| **Elemento (Element)** | Nodo que corresponde a una etiqueta. Los elementos tienen atributos, clases, hijos y métodos específicos como querySelector() o closest(). | `document.querySelector("main") instanceof Element` |
| **Nodo de texto** | El texto entre etiquetas también forma nodos. Los saltos de línea e indentaciones del HTML pueden producir nodos de texto con espacios. | `document.body.firstChild // puede ser #text` |
| **Padre e hijo** | parentNode/parentElement apuntan hacia arriba; childNodes/children permiten bajar. Element.parentElement devuelve un elemento o null. | `const padre = item.parentElement;` |
| **Hermanos** | nextSibling/previousSibling recorren cualquier nodo; nextElementSibling/previousElementSibling saltan directamente entre elementos. | `fila.nextElementSibling` |
| **Contenido textual** | textContent obtiene o sustituye el texto de un nodo y descendientes sin interpretar HTML. Es la opción segura para texto normal. | `mensaje.textContent = "Operación completada";` |

## 3. Tipos de nodo y propiedades fundamentales

| CONCEPTO | EXPLICACIÓN / QUÉ RECORDAR | EJEMPLO |
|---|---|---|
| **nodeType** | Número que identifica el tipo de nodo. Es preferible comparar con constantes como Node.ELEMENT_NODE o Node.TEXT_NODE en vez de memorizar números. | `if (nodo.nodeType === Node.TEXT_NODE) { ... }` |
| **Element node** | Representa etiquetas HTML/SVG. Node.ELEMENT_NODE vale 1. | `elemento.nodeType === Node.ELEMENT_NODE` |
| **Text node** | Contiene texto y no tiene elementos hijos. Node.TEXT_NODE vale 3. | `nodo.nodeType === Node.TEXT_NODE` |
| **Comment node** | Los comentarios HTML también son nodos. Normalmente no intervienen en las prácticas, pero forman parte del árbol. | `nodo.nodeType === Node.COMMENT_NODE` |
| **Attr** | Los atributos pueden representarse mediante objetos Attr, pero no son hijos del elemento dentro del árbol DOM moderno. Normalmente se gestionan con getAttribute/setAttribute o propiedades. | `entrada.getAttribute("type")` |
| **nodeName** | Nombre del nodo. En elementos HTML suele aparecer en mayúsculas; en texto se obtiene #text. | `document.body.nodeName // "BODY"` |
| **nodeValue** | Valor de determinados nodos como Text o Comment. En elementos suele ser null; para texto visible se prefiere textContent. | `texto.nodeValue = "Nuevo texto";` |

## 4. Recorrido del árbol: hijos, padres y hermanos

| CONCEPTO | EXPLICACIÓN / QUÉ RECORDAR | EJEMPLO |
|---|---|---|
| **childNodes** | NodeList viva con todos los nodos hijo, incluidos textos y comentarios. Puede sorprender si el HTML tiene espacios o saltos. | `console.log(panel.childNodes.length);` |
| **children** | HTMLCollection viva con sólo elementos hijo. Suele ser más cómoda cuando sólo importan etiquetas. | `for (const hijo of panel.children) { ... }` |
| **firstChild / lastChild** | Devuelven el primer/último nodo de cualquier tipo. Pueden ser nodos de texto de espacios. | `document.body.firstChild` |
| **firstElementChild / lastElementChild** | Devuelven únicamente elementos. Son preferibles cuando se trabaja con etiquetas HTML. | `document.body.firstElementChild` |
| **nextSibling / previousSibling** | Avanzan entre nodos de cualquier tipo. | `nodo.nextSibling` |
| **nextElementSibling / previousElementSibling** | Avanzan entre elementos ignorando nodos de texto/comentario. | `fila.previousElementSibling` |
| **Recorrido recursivo** | Una función puede visitar un nodo y llamar a la misma función para cada hijo. Es útil para búsquedas profundas o transformación de textos. | `function visitar(n){ for (const h of n.childNodes) visitar(h); }` |

| CONCEPTO | EXPLICACIÓN / QUÉ RECORDAR | EJEMPLO |
|---|---|---|
| **TreeWalker** | API especializada para recorrer determinados tipos de nodo. Es muy útil cuando se desea, por ejemplo, visitar sólo nodos de texto. | `const w = document.createTreeWalker(document.body, NodeFilter.SHOW_TEXT);` |

**PARA LAS PRÁCTICAS** El recorrido de nodos de texto se utiliza en algunos ejercicios. Es importante distinguir textContent de innerHTML y evitar modificar etiquetas cuando sólo se pretende cambiar texto.

## 5. Selección de elementos: API clásica y selectores CSS

| CONCEPTO | EXPLICACIÓN / QUÉ RECORDAR | EJEMPLO |
|---|---|---|
| **getElementById()** | Busca un elemento por id. Devuelve el elemento o null. El id debe ser único en el documento. | `const info = document.getElementById("info");` |
| **getElementsByTagName()** | Devuelve una HTMLCollection viva de elementos con una etiqueta concreta. | `const enlaces = document.getElementsByTagName("a");` |
| **getElementsByClassName()** | Devuelve una HTMLCollection viva de elementos que contienen la clase indicada. | `const avisos = document.getElementsByClassName("aviso");` |
| **querySelector()** | Acepta selectores CSS y devuelve la primera coincidencia o null. Es muy flexible para buscar por id, clase, atributos o estructura. | `const segundo = document.querySelector("ul > li:nth-child(2)");` |
| **querySelectorAll()** | Acepta selectores CSS y devuelve una NodeList estática con todas las coincidencias. | `const botones = document.querySelectorAll("button.accion");` |
| **Selector dentro de un elemento** | querySelector/querySelectorAll también existen en Element, lo que permite limitar la búsqueda a una zona. | `tarjeta.querySelector(".precio")` |
| **matches()** | Comprueba si un elemento satisface un selector CSS. | `if (item.matches(".pendiente")) { ... }` |
| **closest()** | Sube por los ancestros hasta localizar el primer elemento que coincida con el selector. Muy útil con eventos. | `const fila = evento.target.closest(".fila");` |

## 6. Colecciones DOM: NodeList, HTMLCollection y Array

| CONCEPTO | EXPLICACIÓN / QUÉ RECORDAR | EJEMPLO |
|---|---|---|
| **HTMLCollection** | Colección de elementos. Las obtenidas con getElementsByTagName() y getElementsByClassName() son vivas: reflejan cambios posteriores del DOM. | `const filas = tabla.getElementsByTagName("tr");` |
| **NodeList estática** | querySelectorAll() devuelve una fotografía de las coincidencias en ese momento. Añadir nodos después no cambia esa NodeList. | `const antes = document.querySelectorAll("li");` |
| **NodeList viva** | Algunas propiedades, como childNodes, devuelven NodeList vivas. No debe generalizarse que “toda NodeList es estática”. | `const nodos = panel.childNodes;` |
| **No son Arrays** | NodeList y HTMLCollection son iterables/array-like, pero no poseen todos los métodos de Array. | `// colección.map(...) puede fallar` |
| **Conversión a Array** | Array.from() o spread permiten obtener un Array real y usar map/filter/reduce. | `const lista = Array.from(document.querySelectorAll("li"));` |
| **Iteración** | for...of funciona en colecciones modernas. querySelectorAll() también ofrece forEach(). | `for (const boton of botones) { ... }` |

## 7. Lectura y modificación de texto y contenido

| CONCEPTO | EXPLICACIÓN / QUÉ RECORDAR | EJEMPLO |
|---|---|---|
| **textContent** | Lee o escribe texto sin interpretar etiquetas. Es la elección normal cuando el contenido es texto o procede del usuario. | `salida.textContent = nombreUsuario;` |
| **innerHTML** | Interpreta una cadena como HTML. Es cómodo para contenido fijo y controlado, pero peligroso con datos no confiables. | `panel.innerHTML = "<strong>Listo</strong>";` |

| CONCEPTO | EXPLICACIÓN / QUÉ RECORDAR | EJEMPLO |
|---|---|---|
| **innerText** | Representa texto “renderizado” y depende de estilos/visibilidad. Puede forzar cálculos de layout; no suele ser la primera opción para lógica. | `console.log(elemento.innerText);` |
| **value** | Propiedad habitual de input, textarea y otros controles de formulario. | `const nombre = inputNombre.value.trim();` |
| **document.title** | Permite consultar/modificar el título de la pestaña. | `document.title = "Gestor de material";` |
| **document.links** | Colección de enlaces <a>/<area> con href. Puede ser útil para contar enlaces en ejercicios básicos. | `console.log(document.links.length);` |

## 8. Creación e inserción de nodos

| CONCEPTO | EXPLICACIÓN / QUÉ RECORDAR | EJEMPLO |
|---|---|---|
| **createElement()** | Crea un elemento desconectado del documento. No se verá hasta insertarlo. | `const li = document.createElement("li");` |
| **createTextNode()** | Crea explícitamente un nodo de texto. Es útil para comprender el árbol y evitar interpretar HTML. | `li.appendChild(document.createTextNode("Tema 3"));` |
| **appendChild()** | Añade un nodo como último hijo y devuelve el nodo añadido. Si el nodo ya estaba en el DOM, se mueve. | `lista.appendChild(li);` |
| **append()** | Método moderno que acepta varios nodos y/o cadenas. No devuelve el nodo insertado. | `lista.append(li, "fin");` |
| **prepend()** | Inserta al principio del contenido de un elemento. | `panel.prepend(aviso);` |
| **before() / after()** | Insertan hermanos antes o después del elemento. Evitan tener que calcular manualmente el hermano siguiente. | `referencia.after(nuevoElemento);` |
| **insertBefore()** | Método clásico de Node: inserta antes de un hijo de referencia. Para insertar al final se puede usar null. | `padre.insertBefore(nuevo, referencia);` |
| **DocumentFragment** | Contenedor temporal ligero para preparar varios nodos antes de insertarlos juntos. | `const f=document.createDocumentFragment(); f.append(a,b,c); lista.append(f);` |

**ACTUALIZACIÓN** En código moderno existe Element.after(), por lo que ya no es cierto que “no haya método para insertar después”. Aun así, implementar manualmente insertAfter con parentNode/nextSibling es un buen ejercicio para comprender el árbol DOM.

## 9. Mover, sustituir, eliminar y clonar elementos

| CONCEPTO | EXPLICACIÓN / QUÉ RECORDAR | EJEMPLO |
|---|---|---|
| **Mover un nodo existente** | Insertar un nodo que ya pertenece al documento lo traslada: no se duplica. Esto permite pasar tareas entre columnas sin recrearlas. | `acabadas.appendChild(tarea);` |
| **replaceWith()** | Método moderno para reemplazar un elemento por otro(s). | `antiguo.replaceWith(nuevo);` |
| **replaceChild()** | Método clásico del padre: replaceChild(nuevo, antiguo). | `contenedor.replaceChild(nuevo, antiguo);` |
| **remove()** | Elimina directamente el elemento del DOM. | `fila.remove();` |
| **removeChild()** | Método clásico del padre para quitar un hijo. Devuelve el nodo eliminado. | `padre.removeChild(hijo);` |
| **cloneNode()** | Copia un nodo. Con true copia descendientes; con false sólo el nodo. Los listeners añadidos con addEventListener no se clonan. | `const copia = tarjeta.cloneNode(true);` |
| **replaceChildren()** | Sustituye todos los hijos por los nodos/cadenas indicados; sin argumentos vacía el contenedor. | `lista.replaceChildren();` |

## 10. Atributos HTML, propiedades DOM y data-*

| CONCEPTO | EXPLICACIÓN / QUÉ RECORDAR | EJEMPLO |
|---|---|---|
| **getAttribute()** | Lee el valor textual de un atributo HTML. | `const alt = imagen.getAttribute("alt");` |
| **setAttribute()** | Crea o modifica un atributo. | `imagen.setAttribute("alt", "Osciloscopio");` |
| **removeAttribute()** | Elimina un atributo. | `boton.removeAttribute("disabled");` |
| **toggleAttribute()** | Añade o elimina un atributo booleano. Puede recibir un segundo argumento para forzar estado. | `panel.toggleAttribute("hidden");` |
| **Propiedad DOM** | Muchos atributos se reflejan como propiedades del objeto. Para controles, propiedades como value, checked o disabled son especialmente útiles. | `casilla.checked = true;` |
| **Atributo vs propiedad** | El atributo representa el valor en el marcado; la propiedad puede reflejar el estado actual del objeto. En formularios pueden divergir tras interacción. | `console.log(input.value);` |
| **data-* / dataset** | Permite asociar metadatos propios sin inventar atributos no estándar. data-id-tarea se lee como dataset.idTarea. | `fila.dataset.idTarea = "42";` |

## 11. CSS desde JavaScript: style, className y classList

| CONCEPTO | EXPLICACIÓN / QUÉ RECORDAR | EJEMPLO |
|---|---|---|
| **style** | Manipula estilos en línea. Las propiedades con guion se expresan en camelCase. Útil para valores calculados puntuales. | `panel.style.backgroundColor = "#eef";` |
| **getComputedStyle()** | Consulta el estilo efectivo tras aplicar CSS, herencia y reglas del navegador. | `const color = getComputedStyle(panel).color;` |
| **className** | Lee o reemplaza la cadena completa de clases. Puede borrar clases existentes si se asigna sin cuidado. | `panel.className = "tarjeta activa";` |
| **classList.add()** | Añade una o varias clases sin eliminar las existentes. | `panel.classList.add("destacado");` |
| **classList.remove()** | Elimina una clase. | `panel.classList.remove("oculto");` |
| **classList.toggle()** | Alterna una clase; devuelve true si queda presente. Es ideal para mostrar/ocultar o cambiar estado visual. | `panel.classList.toggle("oculto");` |
| **classList.contains()** | Comprueba si una clase está presente. | `if (panel.classList.contains("prestado")) { ... }` |
| **classList.replace()** | Sustituye una clase por otra. | `tarea.classList.replace("pendiente","hecha");` |
| **visibility / display / hidden** | visibility:hidden oculta pero conserva espacio; display:none retira del layout; el atributo hidden es otra opción semántica para ocultar. | `panel.style.visibility = "hidden";` |

**BUENA PRÁCTICA** Para estados visuales repetibles, define clases en CSS y alterna esas clases con classList. Reserva style para valores realmente dinámicos o demostraciones específicas. Así se mantiene la separación entre aspecto y comportamiento.

## 12. Carga del documento y ubicación del código JavaScript

| CONCEPTO | EXPLICACIÓN / QUÉ RECORDAR | EJEMPLO |
|---|---|---|
| **Script al final de body** | Una solución clásica para asegurarse de que los elementos anteriores ya existen cuando se ejecuta el script. | `... <script src="app.js"></script> </body>` |
| **defer** | Descarga el script en paralelo y lo ejecuta después de analizar el HTML, respetando el orden de scripts defer. | `<script src="app.js" defer></script>` |
| **type="module"** | Los módulos JavaScript se difieren por defecto y permiten import/export. En proyectos modernos es habitual usar módulos/bundlers. | `<script type="module" src="main.js"></script>` |
| **DOMContentLoaded** | Evento que indica que el HTML se ha analizado y el DOM está construido, sin esperar necesariamente a todas las imágenes. | `document.addEventListener("DOMContentLoaded", iniciar);` |

| CONCEPTO | EXPLICACIÓN / QUÉ RECORDAR | EJEMPLO |
|---|---|---|
| **load** | Se dispara cuando el documento y recursos dependientes han terminado de cargar. Suele ser más tarde que DOMContentLoaded. | `window.addEventListener("load", medirImagenes);` |

## 13. Eventos del DOM: fundamentos

| CONCEPTO | EXPLICACIÓN / QUÉ RECORDAR | EJEMPLO |
|---|---|---|
| **Evento** | Suceso producido por el usuario, navegador o código: click, input, mouseout, load, etc. En UT03 se usan para activar modificaciones del DOM; UT04 profundiza en ellos. | `boton.addEventListener("click", guardar);` |
| **addEventListener()** | Método recomendado para registrar manejadores sin mezclar comportamiento con HTML y permitiendo varios listeners. | `boton.addEventListener("click", () => contador++);` |
| **Propiedad onclick** | Asigna un único manejador desde JavaScript. Es válida, pero menos flexible que addEventListener. | `boton.onclick = guardar;` |
| **Atributo onclick** | Código en el HTML. Algunas prácticas heredadas lo exigen, pero en código moderno se prefiere separar HTML y JavaScript. | `<button onclick="borrar(7)">Borrar</button>` |
| **Objeto event** | El navegador pasa información del evento al manejador. | `function manejar(event){ console.log(event.type); }` |
| **target** | Elemento donde se originó el evento. | `console.log(event.target);` |
| **currentTarget** | Elemento cuyo listener se está ejecutando. Puede diferir de target durante propagación. | `console.log(event.currentTarget);` |
| **preventDefault()** | Evita la acción predeterminada cuando proceda, por ejemplo el envío de un formulario o navegación de un enlace. | `event.preventDefault();` |

**EXCEPCIÓN DIDÁCTICA** Se puede usar onClick/onclick con un identificador, pero el patrón recomendado para proyectos nuevos es addEventListener() o, en React, los manejadores JSX.

## 14. Propagación y delegación de eventos

| CONCEPTO | EXPLICACIÓN / QUÉ RECORDAR | EJEMPLO |
|---|---|---|
| **Bubbling** | La mayoría de eventos se propagan desde el elemento origen hacia sus ancestros. Esto permite escuchar muchos hijos desde un único contenedor. | `lista.addEventListener("click", manejarLista);` |
| **Capturing** | Fase previa de propagación desde ancestros hacia el objetivo. Puede activarse con la opción capture. | `panel.addEventListener("click", fn, {capture:true});` |
| **stopPropagation()** | Detiene la propagación. Debe usarse sólo cuando sea necesario porque puede dificultar la composición de componentes. | `event.stopPropagation();` |
| **Delegación** | Un listener en el padre identifica qué descendiente originó el evento. Es especialmente útil con elementos creados dinámicamente. | `const boton=event.target.closest("button[data-accion]");` |
| **dataset en delegación** | Los atributos data-* permiten declarar la acción/id sin generar un listener para cada botón. | `<button data-accion="archivar" data-id="8">...</button>` |

## 15. Seguridad al generar contenido dinámico

| CONCEPTO | EXPLICACIÓN / QUÉ RECORDAR | EJEMPLO |
|---|---|---|
| **Riesgo de innerHTML** | Si se inserta como HTML una cadena controlada por el usuario, podría interpretarse código o marcado no deseado. | `// Evitar: salida.innerHTML = entrada.value;` |
| **Preferir textContent** | Cuando sólo se necesita texto, textContent no interpreta etiquetas y reduce el riesgo de inyección. | `salida.textContent = entrada.value;` |
| **Construcción con createElement** | Para estructuras complejas, crear nodos y asignar textContent mantiene separadas estructura y datos. | `const p=document.createElement("p"); p.textContent=dato;` |
| **Contenido confiable** | innerHTML puede ser razonable con plantillas fijas controladas por el programador; aun así, conviene limitar su uso. | `panel.innerHTML = "<em>Sin resultados</em>";` |
| **URL e imágenes** | Las URLs de datos externos también deben validarse si la aplicación acepta entradas del usuario. | `imagen.src = urlValidada;` |

## 16. Compatibilidad entre navegadores y desarrollo multicliente

| CONCEPTO | EXPLICACIÓN / QUÉ RECORDAR | EJEMPLO |
|---|---|---|
| **Estándares web** | Programar con APIs estandarizadas reduce diferencias entre Firefox, Chrome y otros navegadores. | `document.querySelector(".tarjeta")` |
| **Feature detection** | Se comprueba la capacidad que se necesita, no la marca del navegador. | `if ("IntersectionObserver" in window) { ... }` |
| **CSS.supports()** | Permite comprobar compatibilidad de una característica CSS desde JavaScript. | `if (CSS.supports("display", "grid")) { ... }` |
| **Evitar UA sniffing** | Analizar navigator.userAgent o appVersion es frágil y no garantiza capacidades. Sólo debe plantearse como último recurso. | `// Preferir: "clipboard" in navigator` |
| **Fallback** | Si una característica no existe, se ofrece una alternativa más simple. | `const copiar = navigator.clipboard ? copiarApi : copiarManual;` |
| **Prueba multicliente** | Durante el desarrollo, probar al menos en los dos navegadores instalados en el entorno del curso y revisar consola/inspector. | `firefox index.html // y comprobar también en Chrome` |
| **Baseline/compatibilidad** | Antes de usar una API poco habitual, revisar documentación actual (MDN/compatibilidad) y no asumir soporte sólo porque funcione en un navegador. | `Consulta MDN → “Browser compatibility”` |

## 17. Separación de contenido, presentación y comportamiento

| CONCEPTO | EXPLICACIÓN / QUÉ RECORDAR | EJEMPLO |
|---|---|---|
| **Contenido: HTML** | Define estructura semántica y datos base. Evita incrustar lógica compleja en atributos HTML. | `<section id="avisos"><h2>Avisos</h2></section>` |
| **Presentación: CSS** | Define apariencia y estados visuales mediante clases. | `.oculto { display:none; }` |
| **Comportamiento: JavaScript** | Selecciona elementos, responde a eventos y cambia datos/estructura/clases. | `panel.classList.toggle("oculto");` |
| **Acoplamiento bajo** | Cuanto menos dependa el JS de detalles visuales y el HTML de código JS, más fácil resulta mantener la aplicación. | `data-accion="borrar" + listener delegado` |
| **Nombres semánticos** | Usar clases/ids que expresen función o estado, no sólo apariencia, facilita cambios de diseño. | `class="tarea completada" // mejor que class="verde"` |
| **Documentación del código** | Las funciones deben expresar intención y comentar decisiones relevantes, no explicar sintaxis obvia. | `function moverATerminadas(idTarea) { ... }` |

## 18. Patrones útiles

| CONCEPTO | EXPLICACIÓN / QUÉ RECORDAR | EJEMPLO |
|---|---|---|
| **Contar elementos** | Seleccionar una colección y consultar length. Para enlaces también existe document.links. | `const total = document.querySelectorAll("p").length;` |
| **Leer un elemento concreto** | Las colecciones usan índice desde 0. Antes de acceder conviene comprobar que existe. | `const segundo = document.querySelectorAll("p")[1];` |
| **Añadir <li> dinámico** | Crear elemento, asignar textContent e insertarlo en la lista. | `const li=document.createElement("li"); li.textContent=String(Math.trunc(Math.random()*1000)+1); ul.append(li);` |
| **Insertar una fila** | Crear tr/td o usar insertRow/insertCell; mantener los datos con textContent. | `const tr=document.createElement("tr"); const td=document.createElement("td"); tr.append(td);` |
| **insertAfter manual** | Se puede implementar con parentNode e insertBefore usando nextSibling; si no hay siguiente, se añade al final. | `function insertAfter(nuevo, ref){ ref.parentNode.insertBefore(nuevo, ref.nextSibling); }` |

| CONCEPTO | EXPLICACIÓN / QUÉ RECORDAR | EJEMPLO |
|---|---|---|
| **Modificar texto profundo** | Recorrer nodos de texto y sustituir sólo su contenido, sin destruir elementos descendientes. | `if (n.nodeType===Node.TEXT_NODE) { n.nodeValue=... }` |
| **Tabla generada** | Usar bucles para crear filas/celdas y DocumentFragment para agrupar inserciones. | `for(let f=0;f<10;f++){ const tr=document.createElement("tr"); ... }` |
| **Cambiar clases por condición** | Separar el cálculo lógico del cambio visual. | `if (esPrimo(n)) td.classList.add("primo");` |
| **Mover tareas** | append/appendChild mueve el mismo nodo entre contenedores, conservando su identidad. | `acabadas.append(tarea);` |
| **Ocultar/mostrar** | Usar clase o propiedad adecuada. Para “ocupar espacio pero no verse”, visibility es apropiada. | `parrafo.classList.add("invisible");` |
| **Carrusel temporizado** | setInterval puede cambiar src/clases/estilos del elemento. Conviene guardar el identificador del temporizador si se necesita detenerlo. | `const id=setInterval(siguienteImagen,2000);` |

## 19. React y el DOM: modelo declarativo y reconciliación

| CONCEPTO | EXPLICACIÓN / QUÉ RECORDAR | EJEMPLO |
|---|---|---|
| **React declarativo** | En React se describe qué interfaz corresponde a props/estado; React se ocupa de mantener el DOM sincronizado. No se seleccionan nodos para cada cambio normal. | `return <p>{mensaje}</p>;` |
| **Render** | React ejecuta componentes para calcular el resultado que debería mostrarse. Renderizar no significa necesariamente modificar inmediatamente cada nodo del DOM. | `function Estado({ok}) { return <span>{ok?"OK":"Pendiente"}</span>; }` |
| **Commit** | Tras calcular cambios, React aplica al DOM sólo las actualizaciones necesarias para reflejar el resultado. | `estado cambia → render → commit` |
| **Reconciliación** | React compara la nueva descripción de la interfaz con la anterior para decidir qué conservar y qué actualizar. Las keys ayudan a identificar elementos de listas. | `items.map(i => <Fila key={i.id} ... />)` |
| **DOM real** | Es el árbol que mantiene el navegador y que se observa en DevTools. React finalmente trabaja sobre ese DOM, aunque la programación habitual sea declarativa. | `DevTools → Inspector/Elements` |
| **Evitar manipulación manual** | Modificar directamente nodos que React controla puede entrar en conflicto con renderizados posteriores. Preferir props/estado para cambios de interfaz. | `// Preferir setVisible(v=>!v) frente a remove()` |
| **Escape hatch** | Refs permiten acceso imperativo cuando el navegador exige una operación concreta: foco, scroll, medición, selección, etc. | `inputRef.current.focus();` |

**PRECISIÓN TÉCNICA** No es correcto resumir React como “redibuja todo el DOM y luego lo reemplaza”. React calcula una nueva descripción de la UI y, durante el commit, actualiza el DOM donde sea necesario. El término “Virtual DOM” es útil como modelo mental, pero no sustituye esta idea de render + reconciliación + commit.

## 20. Listas, datos, map() y key en React

| CONCEPTO | EXPLICACIÓN / QUÉ RECORDAR | EJEMPLO |
|---|---|---|
| **Datos externos / JSON** | Los datos pueden importarse o recibirse como props y transformarse en componentes. Es mejor separar datos de presentación. | `import datos from "./datos/modulos.json";` |
| **map()** | Transforma cada elemento de un array en JSX. | `modulos.map(m => <Tarjeta key={m.id} modulo={m} />)` |
| **key** | Identificador estable entre hermanos que ayuda a React a relacionar elementos entre renderizados. | `<li key={recurso.id}>{recurso.nombre}</li>` |
| **Key estable** | Preferir identificadores propios de los datos. No generar una key aleatoria dentro de cada render porque cambiaría continuamente. | `// Bien: key={persona.id} // Evitar: key={crypto.randomUUID()}` |
| **Renderizado condicional** | Un ternario permite mostrar un mensaje cuando la lista está vacía. | `items.length===0 ? <p>Sin datos</p> : items.map(...)` |
| **Componentes pequeños** | Dividir la interfaz permite que cada componente reciba sólo los datos necesarios. | `<Lista recursos={recursos} />` |

| CONCEPTO | EXPLICACIÓN / QUÉ RECORDAR | EJEMPLO |
|---|---|---|
| **CSS por componente** | Puede importarse un CSS específico desde el componente para mantener cercanos estructura y estilo. | `import "./Lista.css";` |

## 21. useRef y acceso controlado al DOM en React

| CONCEPTO | EXPLICACIÓN / QUÉ RECORDAR | EJEMPLO |
|---|---|---|
| **useRef()** | Hook que crea un objeto estable con una propiedad current. Cambiar current no provoca un render. | `const inputRef = useRef(null);` |
| **Asignar ref a DOM** | La prop ref conecta el objeto con el nodo DOM correspondiente después del commit. | `<input ref={inputRef} />` |
| **Acceso mediante current** | Cuando el nodo existe, current apunta al elemento DOM. Antes del montaje puede ser null. | `inputRef.current?.focus();` |
| **Casos adecuados** | Foco, scroll, medición, selección de texto, integración con APIs de navegador o librerías imperativas. | `panelRef.current?.scrollIntoView({behavior:"smooth"});` |
| **Cambiar clase con ref** | En una práctica específica puede alternarse una clase del nodo. Si el estado visual forma parte de la UI, normalmente se prefiere estado React. | `panelRef.current?.classList.toggle("oculto");` |
| **No destruir DOM gestionado** | Evitar remove(), innerHTML destructivo u otras mutaciones que cambien la estructura que React cree controlar. | `// Evitar: ref.current.remove();` |
| **Ref como prop (React 19)** | En React 19 un componente funcional puede recibir ref como prop y pasarlo a un nodo descendiente; forwardRef deja de ser necesario para código nuevo. | `function Campo({ref}) { return <input ref={ref}/>; }` |
| **Estado vs ref** | useState representa información que debe participar en el render; useRef guarda una referencia/valor mutable que no necesita redibujar. | `const [visible,setVisible]=useState(false);` |

## 22. React Router actual: rutas, enlaces y navegación

| CONCEPTO | EXPLICACIÓN / QUÉ RECORDAR | EJEMPLO |
|---|---|---|
| **SPA y routing cliente** | Una SPA puede cambiar de vista y URL sin recargar todo el documento. El router coordina URL, historial y componentes. | `/ → Inicio; /modulos → Modulos` |
| **Instalación actual (v8)** | En React Router 8 se instala react-router. El paquete react-router-dom se eliminó en v8. | `npm i react-router` |
| **BrowserRouter** | Router declarativo que usa la History API del navegador. Puede envolver la aplicación o el árbol de rutas. | `import { BrowserRouter } from "react-router";` |
| **Routes** | Contenedor que selecciona la Route que coincide con la ubicación actual. | `<Routes> ... </Routes>` |
| **Route** | Asocia un patrón de URL con un elemento React. | `<Route path="/modulos" element={<Modulos />} />` |
| **Link** | Navega dentro de la aplicación sin recargar el documento como haría una navegación HTML completa. | `<Link to="/modulos">Módulos</Link>` |
| **NavLink** | Similar a Link, pero facilita estilos basados en si la ruta está activa. | `<NavLink to="/inicio">Inicio</NavLink>` |
| **Ruta comodín** | path="*" permite mostrar una vista de error/no encontrada. | `<Route path="*" element={<Error404 />} />` |

**COMPATIBILIDAD CON MATERIAL HEREDADO** Los enunciados antiguos hablan de react-router-dom. En React Router 8 ese paquete ya no existe: con el entorno actualizado que usamos en este curso se debe utilizar react-router e importar BrowserRouter, Routes, Route, Link, Outlet, useParams, useNavigate, etc. desde "react-router".

## 23. Rutas anidadas, Outlet, parámetros y navegación programática

| CONCEPTO | EXPLICACIÓN / QUÉ RECORDAR | EJEMPLO |
|---|---|---|
| **Layout route** | Un componente de estructura fija puede contener cabecera, navegación, main y pie; Outlet marca dónde aparece la ruta hija. | `function Layout(){ return <><Header/><Nav/><main><Outlet/></main><Footer/></>; }` |
| **Ruta anidada** | Una Route puede contener rutas hijas. Sus paths relativos se añaden al path del padre. | `<Route path="modulos" element={<ModulosLayout/>}>...</Route>` |

| CONCEPTO | EXPLICACIÓN / QUÉ RECORDAR | EJEMPLO |
|---|---|---|
| **Outlet** | Renderiza el elemento de la ruta hija que ha coincidido. | `import { Outlet } from "react-router";` |
| **Ruta index** | Ruta hija por defecto del padre. No necesita path. | `<Route index element={<Resumen />} />` |
| **Parámetro dinámico** | Un segmento :nombre captura una parte variable de la URL. | `<Route path="modulos/:codigo" element={<DetalleModulo/>} />` |
| **useParams()** | Devuelve los parámetros dinámicos de la ruta actual. | `const { codigo } = useParams();` |
| **useNavigate()** | Devuelve una función para navegar como resultado de una acción. | `const navigate=useNavigate(); navigate("/modulos");` |
| **useLocation()** | Permite consultar la ubicación actual gestionada por el router. | `const location=useLocation(); console.log(location.pathname);` |
| **Rutas relativas** | Dentro de rutas anidadas, links y paths relativos evitan repetir la ruta completa y facilitan reorganizar la aplicación. | `<Link to="estadisticas">Estadísticas</Link>` |

## 24. Resumen comparativo

| CONCEPTO | EXPLICACIÓN / QUÉ RECORDAR | EJEMPLO |
|---|---|---|
| **Node vs Element** | Node es la interfaz general; Element representa etiquetas. No todo nodo es elemento. | `Text es Node, pero no Element.` |
| **childNodes vs children** | childNodes incluye texto/comentarios; children sólo elementos. | `body.childNodes ≠ body.children` |
| **firstChild vs firstElementChild** | firstChild puede ser #text por espacios; firstElementChild salta directamente a una etiqueta. | `document.body.firstElementChild` |
| **querySelector vs querySelectorAll** | querySelector → primera coincidencia/null; querySelectorAll → NodeList estática. | `document.querySelectorAll(".fila")` |
| **HTMLCollection vs NodeList** | HTMLCollection clásica suele ser viva; NodeList de querySelectorAll es estática. childNodes es una excepción: NodeList viva. | `getElementsByTagName vs querySelectorAll` |
| **textContent vs innerHTML** | textContent trata texto; innerHTML interpreta marcado y exige cautela de seguridad. | `salida.textContent = usuario` |
| **appendChild vs append** | appendChild admite un nodo y devuelve ese nodo; append admite varios nodos/cadenas y no devuelve valor útil. | `lista.append(li1, li2)` |
| **Mover vs clonar** | Insertar un nodo existente lo mueve; cloneNode crea una copia. | `destino.append(origen) // mueve` |
| **style vs classList** | style cambia CSS inline; classList activa reglas CSS reutilizables y mantiene separación de capas. | `elemento.classList.toggle("activo")` |
| **onclick vs addEventListener** | onclick es simple y único; addEventListener es flexible, admite múltiples listeners/opciones y separa mejor capas. | `boton.addEventListener("click", fn)` |
| **DOM clásico vs React** | DOM clásico suele modificar nodos directamente; React describe UI desde datos y sincroniza el DOM. | `DOM: el.textContent=... / React: return <p>{dato}</p>` |
| **useState vs useRef** | useState participa en render; useRef conserva referencia/valor mutable sin provocar render. | `inputRef.current.focus()` |
| **Link vs <a> en SPA** | Link realiza navegación cliente gestionada por el router; <a> normal provoca navegación del navegador y puede recargar la aplicación. | `<Link to="/modulos">...</Link>` |
| **Key estable vs aleatoria** | Una key debe identificar el mismo elemento entre renderizados; una UUID nueva en cada render rompe esa estabilidad. | `key={item.id}` |
| **Compatibilidad** | Preferir estándares y detección de características frente a detectar navegador/versión. | `"clipboard" in navigator` |

## 25. Bibliografía y documentación de consulta

Las siguientes referencias se incluyen como documentación de consulta y ampliación. El contenido, la estructura y los ejemplos de este material han sido redactados específicamente para este documento y no reproducen textos, figuras ni ejemplos de las fuentes citadas.


- Real Decreto 405/2023, de 29 de mayo (actualización del título y enseñanzas mínimas de DAW). https://www.boe.es/buscar/doc.php?id=BOE-A-2023-13221
- Decreto 114/2025, de 29 de julio, del Consell (currículos de FP en la Comunitat Valenciana), con las modificaciones vigentes. https://dogv.gva.es/datos/2025/08/04/pdf/2025_29742_es.pdf
- MDN Web Docs — Document Object Model (DOM). https://developer.mozilla.org/es/docs/Web/API/Document_Object_Model
- MDN Web Docs — selección y recorrido del árbol DOM. https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model/Selection_and_traversal_on_the_DOM_tree
- WHATWG DOM Standard. https://dom.spec.whatwg.org/
- React — Manipulating the DOM with Refs. https://react.dev/learn/manipulating-the-dom-with-refs
- React — referencia de useRef. https://react.dev/reference/react/useRef
- React Router — instalación declarativa actual. https://reactrouter.com/start/declarative/installation
- React Router — documentación de rutas declarativas. https://reactrouter.com/start/declarative/routing


**NOTA SOBRE VERSIONES** Las APIs web evolucionan y las bibliotecas de terceros cambian con mayor rapidez. En particular, React Router 8 eliminó react-router-dom. Antes de una práctica con dependencias externas conviene comprobar package.json y la documentación oficial de la versión instalada.